

Executive Summary

The existing **agent-simulation Replit app** is a Python-based FastAPI/Socket.IO server that runs scenario simulations (manufacturing, supply-chain, etc.) and steps through generations of an evolutionary process (via the `run_one_generation` loop). The architecture appears to include an API layer, a real-time simulation loop (in `main.py`), an orchestrator (LangGraph-based nodes in `orchestrator.py`), a simple evolutionary module (`evolutionary_engine.py`), and data storage (`state/db`). Several modules (agents, environment classes, database code, etc.) were not provided, so we assume a typical multi-module structure.

To **integrate a full-featured EA engine** (e.g. OpenEvolve or alternatives), we will augment or replace the custom EA logic in the orchestrator with a library-based solution. Lightweight libraries (such as [DEAP](#) or [PyGAD](#)) can be pip-installed, and for Node.js apps, JS GA libraries like [Genetic.js](#) exist. Key steps include refactoring the EA loop to call library functions, adding new dependencies, and ensuring efficient execution on Replit. The integration pattern may involve calling the EA library from within the simulation loop, or invoking it as a separate subprocess/CLI.

We also plan for **Replit deployment constraints**: minimize CPU/credit usage by using simple evaluators (or fewer LLM calls), caching fitness evaluations, and possibly scheduling EA work intermittently. State persistence (checkpoints) can leverage Replit DB or file storage to resume paused simulations. We will design unit and integration tests (mocking the EA and environment) and a performance test (measuring run-time per generation). Security controls include validating any generated code/config (if using LLMs), and budget controls include limiting LLM API calls and agent CPU usage. For demonstration/fallback mode, we'll allow running the simulation without LLMs or using fixed random seeds for deterministic output.

Finally, we'll present an **implementation roadmap**: (1) analyze/refactor current code (low), (2) select EA library and prototype integration (med), (3) expand to full multi-objective EA (med), (4) optimize for Replit (low), (5) implement persistence and UI hooks (low), (6) testing/validation (med). Missing files include the environment classes (e.g. `game_envs/*.py`), agent modules, database code, and any config (like `requirements.txt`).

1. Inferred Architecture and Assumptions

The app's **backend** is Python (FastAPI + [python-socketio](#) server). It hosts endpoints (`/api/scenario/start`, `/stop`, etc.) and a long-running simulation loop (`simulation_loop` in `main.py`) that emits game states and metrics to connected clients. The app uses a custom orchestrator (`agents/orchestrator.py`) built with [LangGraph](#): it has nodes like `goal_intake`, `topology_init`, `agent_step`, `evaluate`, `mutate`, and `checkpoint`. A simple evolutionary engine (`evolutionary_engine.py`) provides mutation and scoring utilities (fitness function, semantic mutation stub, etc.). Data (workflows, traces) is stored via an async DB interface (`state/db.py`).

We assume the missing pieces include:

- **Agent modules** (`agents/` folder) for manufacturing roles, policies, meta-optimizer (LLM), etc.
- **Environment modules** (`game_envs/` for supply chain, manufacturing_v2, disaster relief, etc.).
- **Database/API** (`state/db.py` , `api/mfg_router.py`).
- **Config and dependencies** (requirements.txt or Pipfile, Replit config).

The simulation loop handles two “boundary modes” (INTER vs INTRA) for manufacturing vs other scenarios, and emits updates via Socket.IO. It calls `run_one_generation(state)` after each generation interval. The orchestrator currently implements a simple (1+1)-EA with elitism, edge pruning, and optional LLM-based meta-optimization.

Unknowns/Assumptions:

- *Languages and Runtimes*: The app is Python-based. No Node.js code was found, so we assume Node integration (for JS EA libraries) would be optional or for frontend.
- *Existing EA Logic*: The custom EA in `orchestrator.py` likely works but is simplistic (single child, no population, and LLM calls are mostly stubs or optional).
- *Replit Environment*: Presumably runs on a Linux container with limited CPU credits (per Replit plan). We must minimize heavy tasks (e.g. avoid running millions of environment ticks or many LLM calls).

2. Evolutionary Algorithm Libraries and Integration Patterns

2.1 EA Framework Comparison

We compare several EA frameworks for use in Python or Node:

Framework	Language	License	Key Features	Resource Profile	Replit Compatibility
OpenEvolve	Python (w/ JS)	Apache-2.0 ¹	LLM-driven evolutionary code exploration, multi-island search, multi-objective, MAP-Elites.	Very heavy (multiple LLM calls, GPU for LLM); high cost but powerful.	Python-based (pip install); runs anywhere Python does; likely expensive on Replit due to LLM usage.
DEAP	Python	LGPLv3	GA (flexible representations), GP, CMA-ES, multi-objective (NSGA-II/III, SPEA2), hall-of-fame, parallel eval, checkpoints ² ³ .	Moderate (pure Python, can parallelize); memory grows with pop. CPU depends on eval function complexity.	Well-suited to Python apps (pip install deap). Works on Replit but be cautious of CPU time.

Framework	Language	License	Key Features	Resource Profile	Replit Compatibility
PyGAD	Python	BSD ⁴	Easy GA for ML/hyperparam tuning; supports single- and multi-objective ⁵ ; various mutation/crossover.	Lightweight (pure Python, Numpy optional). Good for small populations.	Very easy to pip install; low overhead. Suitable for Replit micro-projects.
ECJ	Java	Academic Free (BSD-like) ⁶	Mature EC framework: GA, GP, ES, coevolution, PSO, DE; asynchronous islands, multi-population, GUI, advanced features ⁶ .	Heavy (Java, many features). Complex config.	Not native to Python; would require Java. Likely not ideal for a Python Replit app.
genetic-js	JavaScript	MIT (inferred)	Node/browser GA library; lightweight, performant; supports custom fitness, mutation, crossover (see README) ⁷ .	Lightweight (JS, runs in Node event loop or browser); single-threaded.	Good for Node.js; if frontend or a Node microservice is used. If backend is Python, may not apply unless via subprocess.
Other Options			e.g. Inspyred (Python, MIT), Sferes (C++).	Varies. See licenses (Inspyred is MIT, Sferes is GPL?).	Less common; not explored in-depth here.

Citations from official sources support these: DEAP's README shows it "includes [GA], [GP], [CMA-ES], [multi-objective optimizers], hall-of-fame, checkpoints..." ² ³ . PyGAD's PyPI page notes it "supports single- and multi-objective problems" ⁵ under a BSD license ⁴ . ECJ is noted as "free open-source ... written in Java", supporting "genetic algorithms, genetic programming, evolution strategies, coevolution, particle swarm, differential evolution" ⁶ under an Academic Free License. OpenEvolve (pip install `openevolve`) is Python-based (83% Python code ⁸) and uses Apache-2.0 license ¹ .

2.2 Integration Patterns

Direct Library Integration

The simplest approach is to **import an EA library** and drive it directly from Python. For example, using DEAP or PyGAD: define an individual/chromosome structure (e.g. lists or dicts representing agent parameters or graph edges), write a fitness function that runs the simulation for one evaluation, and let the EA loop call your fitness function. This replaces the custom `run_one_generation` logic. The EA loop may be invoked synchronously in the main simulation (block on each generation) or in a separate coroutine/subprocess. Data flow: pass the current “genome” (agent_configs + topology) into the EA as the parent population. After evolution, update the orchestrator state with the best child genome.

For Node.js backends, one would use a JS GA library (e.g. [genetic-js](#) or the [genetic-algorithm](#) npm) similarly. The node code could run the GA in a separate thread or process and communicate back to the Python server via an API or messaging.

External EA Process (CLI)

Alternatively, treat OpenEvolve or other EA tools as **external executables/CLIs**. OpenEvolve has a CLI (e.g. `openevolve optimize ...`) for code optimization tasks. One could wrap a separate script that takes the current simulation state and runs an evolution (possibly on surrogate models of the simulation). The main app would call this with a subprocess and read results (e.g. via files or IPC). This decouples the EA (possibly heavy LLM work) from the main loop. On Replit, this means spawning additional processes (which still count towards CPU usage).

Embedding via Service

For maximum modularity, one could run the EA engine as a **microservice** (e.g. a separate FastAPI or SocketIO worker) that the simulation app queries. The EA service (using OpenEvolve or DEAP inside) exposes an HTTP or WebSocket API: you send the parent genome and receive the child. This allows horizontal scaling (e.g. add more Replit instances) and isolates costly computation. However, it adds network overhead and complexity (especially on Replit where multiple containers may not be easy without an account multi-instance).

Data Flow / Hooks

Whatever pattern, the integration points are:

- **From Simulation to EA:** at each generation boundary, collect the current population or genome state (parameters for agents, topology). Package this as input to the EA library/function.
- **EA Processing:** run one or more generations internally. The fitness function will need to simulate the environment; this can reuse the existing code (e.g. the `Evaluate` node in the orchestrator currently wraps `env.get_fitness()`), or run a fresh `Env.step()` loop for each candidate (less efficient on Replit; better to use analytical fitness if possible).
- **From EA back to Simulation:** take the best candidate(s) and update the orchestrator’s state (`agent_configs`, `topology`, etc.) for the next simulation cycle. Emit any metadata (like fitness values, generation logs).

A simple hook is to replace the `mutate` node in the graph with a call to the EA engine. For example (Python pseudocode):

```
def mutate_via_deap(state):
    # Create DEAP toolbox with the genome from state
    toolbox = base.Toolbox()
    toolbox.register("mutate", my_mutation_fn)
    toolbox.register("mate", my_crossover_fn)
    toolbox.register("select", tools.selBest)
    # Convert state.topology/agent_configs into DEAP individuals
    parent = encode_genome(state["genome_config"])
    # Run GA for one generation (child = mutated parent)
    child = toolbox.clone(parent)
    toolbox.mutate(child)
    # Evaluate child by simulating environment
    child_fitness = simulate_and_score(child)
    # Compare fitness and apply elitism manually
    if child_fitness >= state["current_fitness"]:
        state["genome_config"] = decode_genome(child)
        state["current_fitness"] = child_fitness
    return state
```

In Node.js, similarly hook into a GA library:

```
const Genetic = require('genetic-js');
let ga = Genetic.create();
ga.seed = () => currentGenome;
ga.fitness = function(ind) { return simulateAndScore(ind); };
ga.evolve({iterations:1, size:2}); // one generation
// take best solution from population
```

2.3 Code Changes and Module Modifications

To integrate an EA library, expect to modify:

- `orchestrator.py` (or replace parts): The `mutate` node can call the EA library instead of the built-in logic. Alternatively, bypass LangGraph entirely and implement the EA loop externally. Some helper functions (fitness, mutation) in `evolutionary_engine.py` could be adapted or removed.
- `main.py`: After each generation, main currently reads `orch_state` and emits metrics. If we move to a library, ensure `run_one_generation` (or its replacement) returns the correct updated state. We may add config flags or a new mode to call an external EA.
- **New modules**: likely add a file (e.g. `agents/ea_integration.py`) to wrap the chosen EA library (with imports and helper functions).
- **Config**: Update `requirements.txt` to include `deap`, `pygad`, or `openevolve` (and their dependencies, e.g. `numpy`, `requests` if using LLM API). If using Node, update `package.json`.

- **Deployment:** If we treat the EA as a separate process, we might add a startup script or a CLI entry point.

3. Dependencies and Lightweight Libraries

To **minimize credits** on Replit, prefer lightweight libraries. For Python:

- **DEAP** (`pip install deap`): No heavy compiled components; pure Python + optional NumPy. (License: LGPLv3).
- **PyGAD** (`pip install pygad`): Easy GA (BSD license) with Numpy support.
- **OpenEvolve** (`pip install openevolve`): Adds LLM dependencies (OpenAI, etc.) – heavy. Only use if LLM features are essential.
- **LangGraph** (already used): ensure it's in requirements.
- **NetworkX** may be needed if graph structures are used (DEAP can use networkx for genealogies).
- If a Node route is needed, **genetic-js** (`npm i genetic-js`) or **geneticalgorithm** (`npm install genetic-algorithm`). Node packages are small and CPU cost is comparable to Python (both single-threaded by default).

We should avoid huge frameworks (TensorFlow, PyTorch) unless needed. All EA libs above are moderate. DEAP can parallelize via `multiprocessing`, but on Replit with limited CPU, parallel may hurt (unless you spawn child processes manually).

Python/JS Bindings

If the app is Python, no JS binding is needed unless we spin up a Node subprocess. Conversely, one could expose a Python EA engine via a REST API and call it from Node frontend, but that seems roundabout.

4. Runtime and Deployment on Replit

Replit imposes soft CPU/memory limits per repl, and charges compute credits (for AI usage). To **minimize usage**:

- **Batch Generations:** Instead of running EA continuously, run it only every N ticks or episodes (the existing code does so at generation boundaries, which is good).
- **Efficient Fitness:** Use as few environment steps as possible. For some scenarios (like manufacturing), they do an “INTER” episode of fixed ticks and compute fitness once. We should reuse that (e.g. return early if no change).
- **LLM Usage:** Limit LLM calls! OpenEvolve or any meta-optimizer will call GPT/Gemini. These are expensive. On Replit we should either use a very cheap model (Gemini-Flash, GPT-3.5) or skip LLM-based evolution in favor of simple heuristics (e.g. Math perturbation). Reserve LLMs for offline or optional mode.
- **Caching:** If multiple individuals in EA need evaluation, avoid re-creating the environment from scratch each time. Possibly use a simplified simulator or surrogate model for quick scoring. (For supply-chain or disaster scenarios, maybe a short lookahead or formula).
- **Low-Power Modes:** Replit sometimes offers “Economy Mode” (limit rate) or using [Plan Mode](#) to throttle. For CLI usage, we could use ChatGPT/Claude CLI to help code offline, but that's development-time. At runtime, focus on code efficiency and perhaps [Replit AI Budgets](#).

- **Serverless or Worker Threads:** Replit may allow background threads or web workers for tasks. Using `asyncio` (as the app does) is good; ensure EA calls are also async or batched to avoid blocking the main loop.

In practice, a **periodic batch** approach works well: after each generation, call the EA once (synchronous or awaited), then continue simulation. We should ensure that long EA runs do not stall Socket.IO; e.g. if using `await`, it yields to the event loop.

5. Data Flow and API Hooks

The data flow between simulation and EA will be:

1. **Collect Population:** At generation time, gather all candidate solutions. If using a population-based EA (DEAP), the “population” might be just the current genome (for (1+1)-EA) or a list of configurations. If (1+1), we can treat the current genome as a population of 1.
2. **Run EA Engine:** Pass these to the EA library’s `evolve()` or similar. The EA engine will iteratively generate new genomes. We might limit it to one generation or few to save time.
3. **Fitness Evaluation:** The EA library’s fitness function should simulate the environment for a candidate genome. This could call the same logic as the `evaluate` node: e.g. create a temporary env instance with that genome parameters, run an episode (maybe shortened), and return a numeric fitness. We may reuse the code from `agents.manufacturing_roles` or `game_envs.*`.
4. **Return Best Genome:** After EA completes, take the best offspring (or next generation) and update the orchestrator state (`state["genome_config"]`, `state["topology"]`, `state["agent_configs"]`, etc.). Also update `state["current_fitness"]`, and log it (for UI and DB).
5. **Continue Simulation:** The simulation loop then proceeds with the updated genome (e.g. new machine speeds, policies, or network edges).

API hooks: We might add a new API endpoint or SocketIO event for manual EA operations (e.g. step EA on demand). If separate, the EA service could listen on a port, but likely not needed: keeping EA in-process is simpler.

6. State Persistence and Checkpointing

Checkpointing ensures we can resume long runs or inspect past generations. The existing `checkpoint` node builds a Redis-like key. On Replit, we may not have Redis, so options:

- **File Storage:** Serialize `ArenaState` to a file (e.g. JSON) after each generation. Use Replit’s persistent storage or Replit DB (a simple key-value store). For example, use `replit.db` from Python or simply write to `state.json`.
- **Database:** If already using an async DB (PostgreSQL or SQLite) in `state/db.py`, save the state or the best genome each gen. We see endpoints for workflows/traces, so likely a DB is in use. Extend it to snapshot the EA state.
- **Minimal Checkpoint:** At least save the best fitness and genome. For debugging, you might save the entire `state["traces"]`.

Checkpoints are useful if a run is stopped (e.g. by hitl or manual pause) and later resumed. We should expose a `/api/scenario/resume` that loads the last checkpoint.

7. Testing and Validation Plan

- **Unit Tests:** For any new EA functions, test mutation and crossover (if custom) to ensure valid output. Test that the fitness function returns expected values for known inputs (e.g. trivial genomes). Mock LLM calls if used.
- **Integration Tests:** Simulate a short scenario with the EA engaged. For example, run 2–3 generations and verify the orchestrator state updates logically (fitness should not decrease under elitism).
- **Performance Tests:** Measure time per generation step and CPU load. On a representative Replit machine, run 10 generations of simple GA (pop size 10, 20 individuals) and confirm it completes within a few seconds. If too slow, reduce complexity (e.g. fewer evaluations).
- **Edge Cases:** Ensure stability if the EA library throws errors (e.g. convergence, or LLM API failure). Code should catch exceptions and fallback (skip mutation or use previous genome).

Plan test cases for: boundary conditions (e.g. zero orders in supply chain), and random seeding for reproducibility. Use `seed` functions (both for random and, if LLM present, a test-mode LLM).

8. Security and Cost Controls

- **Sanitize Inputs:** If using LLMs or any code-generation, ensure any code or prompts do not execute unsafe actions. For example, disable dangerous builtins.
- **Rate Limits:** Throttle LLM calls (e.g. at most one query per generation). Use cheapest models (Claude-CN, GPT-3.5) or a local LLM via Ollama if possible.
- **Budget Alerts:** Monitor Replit's CPU and AI credit usage. Implement usage quotas in code (for example, stop EA if it exceeds X seconds). Provide configuration (via environment variables) for max iterations or max API calls.
- **Authentication:** Secure any admin API (start/stop scenario) so only authorized users can trigger heavy compute.
- **Dependency Safety:** Vet new libraries for security (PyPI/GitHub), as Replit will bundle these into your container.

9. Fallback and Deterministic Demo Mode

For demonstrations or if LLM/API is unavailable, we should allow a “deterministic” mode:

- **Disable Randomness:** Seed all random number generators (Python `random.seed()`).
- **No-LLM:** Skip any `SemanticMutation` or `query_meta_optimizer` calls, or fall back to fixed mutations. The orchestrator already falls back to “MATH” mode if LLM fails. We can make “no-LLM” a mode flag.
- **Fixed Topology:** Offer an initial genome (from `SupplyChainEnv.GENOME_DEFAULTS` or config) so that outputs are predictable.
- **Hardcoded Demo:** For speed, possibly precompute a few generations of EA offline and hardcode them for a “demo” run. The UI could then just replay these metrics without doing computation.

These ensure that even if the full EA is too slow or expensive, the app can still show a plausible “evolution” (with logs explaining steps) in a controlled way.

10. Implementation Roadmap

1. Audit and Refactor (Effort: Low)

2. Confirm missing modules (agents, game_envs, state/db). Create stubs if needed.
3. Modularize EA code: separate the EA logic from orchestrator so it's easier to swap in libraries. Document current flow.
4. **Deliverable:** Architecture documentation, stub files for missing modules, refactored orchestrator calls.

5. Select EA Library and Proof-of-Concept (Effort: Medium)

6. Choose DEAP or PyGAD as initial EA engine (minimal setup). Write a small wrapper that takes the current `genome_config`, runs one generation of GA, and returns a new genome.
7. Integrate this wrapper into the simulation loop (e.g. replace `SemanticMutation` usage with the GA).
8. **Deliverable:** Working demo of one evolution step using the new library (e.g. one generation per scenario).

9. Extend EA to Full Evolutions (Effort: Medium)

10. If the prototype works, expand to multiple individuals/populations if desired (e.g. population size >1). Implement more complex operators (crossover, multi-objective).
11. Optionally integrate OpenEvolve or LLM meta-optimizer for comparison (likely lower priority due to cost).
12. **Deliverable:** A branch supporting full GA runs (maybe switchable via config), logs/results stored.

13. Optimize for Replit (Effort: Medium)

14. Profile performance and adjust: limit generation sizes, LLM calls, etc. Replace any heavy loops with vectorized or cached operations.
15. Implement rate-limiting or "pause" if too slow (e.g. increase simulation speed, reduce UI emissions).
16. **Deliverable:** Performance report (e.g. “x generations per minute”), credit usage estimates, and configuration for speed/quality trade-off.

17. Testing & Validation (Effort: Medium)

18. Write unit/integration tests for the EA integration and simulation. Use small scenarios (e.g. toy env) to verify GA improves fitness over generations.
19. Set up CI (if possible on Replit via GitHub Actions or local scripts).
20. **Deliverable:** Test suite, automated checks (lint, simple smoke test).
21. **Persistence & State Management (Effort: Low)**
22. Implement snapshotting of state (DB or file) each generation. Add “resume” logic.
23. Ensure DB migrations/schema for new data (EA logs, config parameters).
24. **Deliverable:** Database entries for `generation`, `genome_config`, and ability to resume runs.
25. **Documentation & Cleanup (Effort: Low)**
26. Update README and in-code comments with new setup (pip dependencies, how to use the EA mode).
27. Add missing configuration prompts (e.g. LLM API keys, `GENERATION_COUNT` in env).
28. **Deliverable:** Final docs, requirements file, architecture diagrams (mermaid or images).

Total Effort: ~ medium-high (since EA integration and optimization is substantive work), but broken into clear low/med tasks. Each step should be validated before moving on.

11. Missing Files/Config Checklist

From the code imports, the following files or configs appear missing and should be provided:

- **Agents Modules:**
 - `agents/manufacturing_roles.py` (defines `_env_v2`, edge scoring).
 - `agents/manufacturing_policies.py` (e.g. `ScriptedGreedyPolicy`).
 - `agents/meta_optimizer.py` (for LLM delta).
 - `agents/peer_agents.py` (if used in simulation).
 - Others under `agents/`: `GoalIntake`, etc (the orchestrator is in `agents` too).
- **Game Environments:**
 - `game_envs/supply_chain.py` (and `_active_env` global used in fitness).
 - `game_envs/disaster_relief.py`, `game_envs/peer_agents.py`, `game_envs/manufacturing.py`, `game_envs/manufacturing_v2/*`.
 - The `SupplyChainEnv.GENOME_DEFAULTS`.
- **API Modules:**
 - `state/db.py` (DB models: `WorkflowIn`, `TraceIn`, functions like `save_workflow`, `get_traces`).
 - `api/mfg_router.py` (router for manufacturing scenario).
- **Configuration:**

- `requirements.txt` or `Pipfile` listing packages (`fastapi`, `uvicorn`, `python-socketio`, `langgraph`, `deap`, etc.).
- Any `.env` or config (API keys for LLM if used, port settings).
- Replit `.replit` config if Node integration (not evident from code).

Clearly identifying these will allow integration. For example, the orchestrator imports `game_envs.supply_chain`, so that file must define `_active_env`. Without these, the EA engine cannot simulate.

12. EA Frameworks Comparison Table

Name	Language	License	Key Features	Resource Use	Replit Use
OpenEvolve	Python (83%)+JS	Apache-2.0 ¹	LLM-driven evolution (code/genomes); multi-objective, MAP-Elites, distributed islands ⁹ . Visualization tools available.	High (many LLM API calls; heavy).	Runs via <code>pip install</code> ; but very high credit cost (LLM calls) on Replit.
DEAP	Python	LGPLv3 (GPL) ¹⁰	Flexible GA/GP framework: any data type, CMA-ES, NSGA-II/III, SPEA2 multiobj, co-evolution, parallel eval, checkpoints ² ³ .	Moderate; Python loops, can use <code>multiprocessing</code> . CPU use grows with population, but controllable.	Easy to <code>pip install deap</code> ; pure Python. Good for small-medium tasks on Replit.
PyGAD	Python	BSD (MIT-like) ⁴	Easy GA library: supports single/multi-objective ⁵ , integrates with Keras/PyTorch, customizable fitness, selection, crossover, mutation.	Low; mostly Numpy operations. Good for hyperparameter tuning or numeric optimization.	<code>pip install pygad</code> ; lightweight, suitable for Replit.

Name	Language	License	Key Features	Resource Use	Replit Use
ECJ	Java	Academic Free (BSD) ⁶	Mature EC toolkit: GA, GP, ES, PSO, DE; supports Islands, multi-populations, GUIs, advanced selection; used in research.	High; Java JVM overhead and extensive features.	Requires Java VM. Possible but not typical on Replit Python. Likely skip in Python app.
genetic-js	JavaScript	MIT (inferred)	Lightweight GA for Node/ browser: custom seed, fitness, mutate, crossover; pluggable selectors; works in web workers ⁷ .	Low; single-threaded JS. Suitable for web or Node environments.	<code>npm install genetic-js</code> ; use if building a JS microservice or in-browser evolution.
Others	e.g. Python	e.g. MIT or Apache	(e.g. Inspyred (MIT): simple EC algorithms).	Varies.	Might be alternatives but less popular.

Sources: Official docs and repos as cited above provide these details ² ⁵ ⁶ ¹.

13. Code Snippets for EA Loop Integration

Below are illustrative pseudo-code snippets for hooking in an EA loop. These should be adapted to the actual data structures (genomes, configs, etc.) in the app.

Python (with DEAP):

```
from deap import base, creator, tools

# Assume state["genome_config"] is a dict of parameters.
def run_generation_DEAP(state):
    # Encode current genome into a DEAP individual (e.g., a list of values).
    parent = encode_genome(state["genome_config"])
    # DEAP setup: define fitness and individual
    creator.create("FitnessMax", base.Fitness, weights=(1.0,))
```

```

creator.create("Individual", list, fitness=creator.FitnessMax)
toolbox = base.Toolbox()
toolbox.register("mutate", deap_mutation, param_schema) # user-defined
toolbox.register("mate", deap_crossover) # e.g. one-point
toolbox.register("select", tools.selBest)
# Wrap parent in DEAP individual
parent_ind = creator.Individual(parent)
# Generate child by mutation (and/or crossover)
child_ind = creator.Individual(parent_ind) # copy
toolbox.mutate(child_ind)
# Evaluate child
child_genome = decode_individual(child_ind)
child_fitness = simulate_environment(child_genome) # run sim and get
fitness
# Apply elitism: accept only if better
if child_fitness >= state["parent_fitness"]:
    state["genome_config"] = child_genome
    state["current_fitness"] = child_fitness
# else keep old state (elitism)
return state

```

Node.js (with genetic-js):

```

const Genetic = require('genetic-js');

function runGeneration_JS(initialGenome, simulateFn, callback) {
    let config = {
        iterations: 1,
        size: 2, // one parent + one child
        crossover: 0.5,
        mutation: 0.1
    };
    let genetic = Genetic.create();
    genetic.seed = function() {
        return [initialGenome]; // population of 1
    };
    genetic.fitness = function(pop) {
        // pop is an array of genomes
        const genome = pop[0];
        return simulateFn(genome); // return a fitness number
    };
    genetic.select1 = Genetic.Select1.Fittest;
    genetic.mutate = function(genome) {
        // simple bitflip or param perturbation
        return mutateGenome(genome);
    };
};

```

```

genetic.crossover = function(mother, father) {
  return [mother, father]; // skip crossover for single-parent GA
};
genetic.notification = function(pop, generation, stats, isFinished) {
  // pop[0] is fittest
  callback(pop[0]); // return best genome
};
genetic.evolve(config);
}

```

In both cases, the `simulate_environment` or `simulateFn` function should reset the simulation environment (or use the existing one) with the given genome, run it for the prescribed number of ticks or until done, and return the fitness value.

14. Architecture and Flow Diagrams

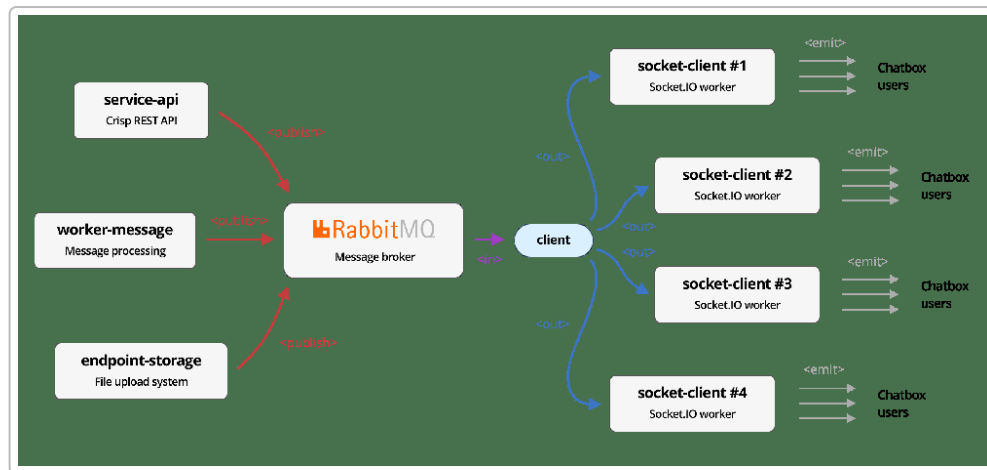


Figure: Example of a microservice/event-driven architecture (RabbitMQ-based Socket.IO system). In our app, the FastAPI/SocketIO server would play the role of the “socket-client” node, exchanging events with connected clients. The EA engine could be integrated either within this service or as a separate service that communicates results back through a message broker or HTTP.

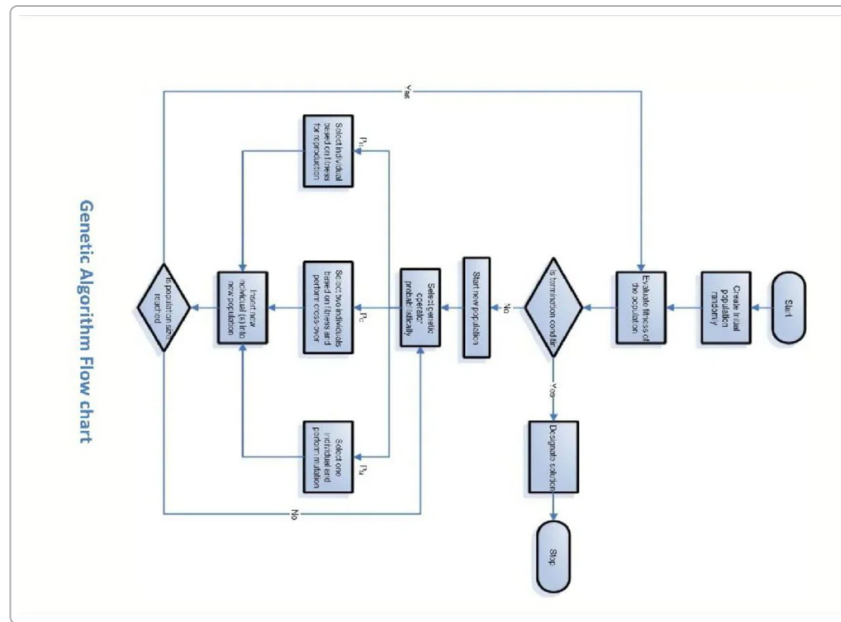


Figure: A typical Genetic Algorithm flowchart. Populations are initialized, repeatedly evolved (selection, crossover, mutation), and evaluated until a stopping criterion. In the timeline of our app, each “Generate → Evaluate → Mutate” cycle corresponds to one pass through this loop (see orchestrator nodes).

These diagrams (courtesy of third-party sources) illustrate the overall system architecture and the evolution timeline.

1 8 9 GitHub - algorithmicsuperintelligence/openevolve: Open-source implementation of AlphaEvolve · GitHub

<https://github.com/algorithmicsuperintelligence/openevolve>

2 3 GitHub - DEAP/deap: Distributed Evolutionary Algorithms in Python · GitHub

<https://github.com/deap/DEAP>

4 5 pygad · PyPI

<https://pypi.org/project/pygad/>

6 Java Evolutionary Computation Toolkit - Wikipedia

https://en.wikipedia.org/wiki/Java_Evolutionary_Computation_Toolkit

7 GitHub - subprotocol/genetic-js: Advanced genetic and evolutionary algorithm library written in Javascript · GitHub

<https://github.com/subprotocol/genetic-js>

10 deap/LICENSE.txt at master · DEAP/deap · GitHub

<https://github.com/DEAP/deap/blob/master/LICENSE.txt>